

POLINOMI E FFT

Martedì 26 Maggio 2026



POLINOMI

- Un polinomio nella variabile x su un campo algebrico F è una funzione $A(x)$ che può essere rappresentata come:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

dove a_j sono i coefficienti ed $n - 1$ è il grado del polinomio (supponiamo che a_{n-1} sia diverso da zero). I coefficienti appartengono al campo F , tipicamente l'insieme $\mathbb{C}$ dei numeri complessi.

- Ogni intero strettamente più grande del grado è detto **grado limite** del polinomio.
- Il grado di un polinomio di grado limite n può essere un qualunque intero tra 0 e $n - 1$



SOMMA DI POLINOMI

- Dati due polinomi $A(x)$ e $B(x)$ di grado limite n , la loro somma è un polinomio $C(x)$, anche di grado limite n , tale che $C(x) = A(x) + B(x)$, per ogni x nel campo F .
- Se $A(x) = \sum_{j=0}^{n-1} a_j x^j$ e $B(x) = \sum_{j=0}^{n-1} b_j x^j$, allora $C(x) = \sum_{j=0}^{n-1} c_j x^j$, dove $c_j = a_j + b_j$



PRODOTTO DI POLINOMI

- Dati due polinomi $A(x)$ e $B(x)$ di grado limite n , il loro prodotto è un polinomio $C(x)$, di grado limite $2n-1$, tale che $C(x) = A(x)B(x)$, per ogni x nel campo F .
- Se $A(x) = \sum_{j=0}^{n-1} a_j x^j$ e $B(x) = \sum_{j=0}^{n-1} b_j x^j$, allora $C(x) = \sum_{j=0}^{2n-2} c_j x^j$, dove $c_j = \sum_{k=0}^j a_k b_{j-k}$
- Si noti che il grado del prodotto è dato dalla somma dei gradi dei due polinomi.
- Se A è un polinomio di grado limite n_a e B ha grado limite n_b , allora C ha grado limite $n_a + n_b - 1$. Quindi $n_a + n_b$ è pure un grado limite per C .



RAPPRESENTAZIONE DI POLINOMI: PER COEFFICIENTI

- La rappresentazione per coefficienti di un polinomio $A(x) = \sum_{j=0}^{n-1} a_j x^j$ di grado limite n è il vettore $(a_0, a_1, \dots, a_{n-1})$. Nelle equazioni matriciali è trattato come vettore colonna
- E' comoda per eseguire la **valutazione di un polinomio $A(x)$ in un dato punto x_0** , ovvero calcolare il valore $A(x_0)$. **Richiede tempo $\Theta(n)$ utilizzando la regola di Horner, ovvero:**

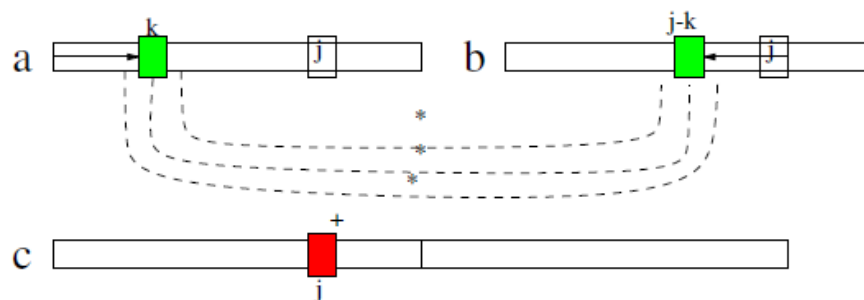
$$A(x_0) = a_0 + x_0(a_1 + x_0(a_2 + \dots + x_0(a_{n-1})) \dots))$$

- La **somma** di due polinomi si ottiene in tempo $\Theta(n)$ sommando i vettori dei coefficienti, elemento per elemento.
- Per quanto riguarda il **prodotto**, se utilizziamo il metodo visto precedentemente sui polinomi rappresentati per coefficienti, si richiede un tempo $\Theta(n^2)$, poiché ciascun coefficiente del vettore di A deve essere moltiplicato per ciascun coefficiente di B



RAPPRESENTAZIONE PER COEFFICIENTI: PRODOTTO DI POLINOMI

- Per quanto riguarda il prodotto, se utilizziamo il metodo visto precedentemente sui polinomi rappresentati per coefficienti, si richiede un tempo $\Theta(n^2)$, poiché ciascun coefficiente del vettore di A deve essere moltiplicato per ciascun coefficiente di B



- Il vettore c dei coefficienti di C è anche detto **convoluzione** di a e b , e indicato con $c = a \otimes b$



RAPPRESENTAZIONE PUNTO VALORE

- La rappresentazione punto-valore (o per punti) di un polinomio $A(x)$ di grado limite n è un insieme di n coppie di valori $\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$ tali che tutti i valori x_k sono tutti distinti e $y_k = A(x_k)$
- Un polinomio ha diverse rappresentazioni per punti, in quanto un insieme qualsiasi di n punti che soddisfa le due proprietà precedenti può essere utilizzato come rappresentazione
- **Dalla rappresentazione per coefficienti è facile calcolare una rappresentazione per punti.** Utilizzando il metodo di Horner, questa valutazione richiede un tempo $\Theta(n^2)$
- **Se gli x_k vengono scelti in modo intelligente questo calcolo può diventare $\Theta(n \log n)$** • • •

SPOILER...attraverso
la FFT!



INTERPOLAZIONE

- L'operazione inversa della valutazione (ovvero, determinare la rappresentazione per coefficienti da una rappresentazione per punti) è detta **interpolazione**.
- **TEOREMA (Unicità del polinomio interpolante):** Dato un insieme qualsiasi di n punti $\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$, tali che tutti i valori x_k siano distinti, esiste un unico polinomio $A(x)$ di grado limite n , tale che $y_k = A(x_k)$ per ogni $k=0, \dots, n-1$.
- La dimostrazione si basa sull'esistenza della inversa della matrice (denotata con $V(x_0, x_1, \dots, x_{n-1})$ e detta matrice di Vandermonde) della seguente equazione:

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}.$$



INTERPOLAZIONE

- Il determinante è $\prod_{0 \leq j < k \leq n-1} (x_k - x_j)$,
- La matrice è invertibile se tutti gli x_k sono distinti.
- I coefficienti si trovano dalla formula:

$$a = V(x_0, x_1, \dots, x_{n-1})^{-1} y.$$

- Si può creare un sistema di equazioni lineari. Tali equazioni possono essere risolte in tempo $\Theta(n^3)$
- Un algoritmo più veloce si basa sulla formula di Lagrange:

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}.$$

- In questo caso, i coefficienti si possono calcolare in tempo $\Theta(n^2)$



OPERAZIONI TRA POLINOMI CON RAPPRESENTAZIONE PUNTO VALORE

- La rappresentazione per punti è comoda per l'operazione di somma tra polinomi. Se $C(x) = A(x) + B(x)$, allora $C(x_k) = A(x_k) + B(x_k)$ per ogni punto x_k .
- Date le seguenti rappresentazioni per punti per A e B
 $\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$ e
 $\{(x_0, y_0'), (x_1, y_1'), \dots, (x_{n-1}, y_{n-1}')\}$,
la rappresentazione per C è la seguente:
 $\{(x_0, y_0 + y_0'), (x_1, y_1 + y_1'), \dots, (x_{n-1}, y_{n-1} + y_{n-1}')\}$
- Quindi, il tempo richiesto per la somma di due polinomi di grado limite n nella rappresentazione per punti è $\Theta(n)$



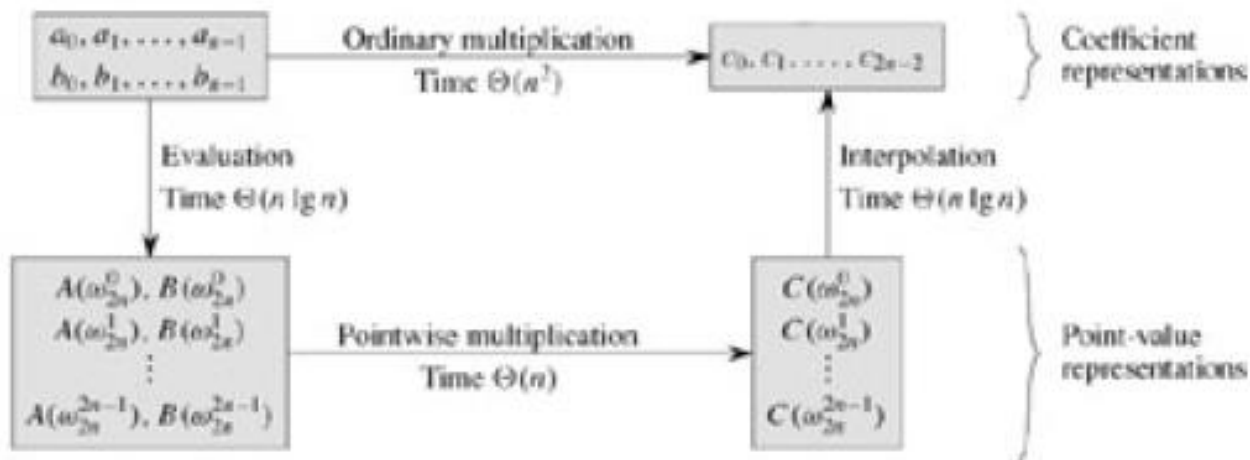
OPERAZIONI TRA POLINOMI CON RAPPRESENTAZIONE PUNTO VALORE

- La rappresentazione per punti è comoda per l'operazione di prodotto tra polinomi. Se $C(x) = A(x)B(x)$, allora $C(x_k) = A(x_k)B(x_k)$ per ogni punto x_k .
- Se A e B hanno grado limite n, allora C ha grado limite 2n, quindi servirebbero 2n coppie punto-valore.
- Quindi, devono essere considerate rappresentazioni per punti «estese» a 2n per A e B.
- Date le seguenti rappresentazioni per punti per A e B
 $\{(x_0, y_0), (x_1, y_1), \dots, (x_{2n-1}, y_{2n-1})\}$ e
 $\{(x_0, y_0'), (x_1, y_1'), \dots, (x_{2n-1}, y_{2n-1}')\}$,
la rappresentazione per C è la seguente:
 $\{(x_0, y_0 y_0'), (x_1, y_1 y_1'), \dots, (x_{2n-1}, y_{2n-1} y_{2n-1}')\}$
- Quindi, il tempo richiesto per il prodotto di due polinomi di grado limite n nella rappresentazione per punti è $\Theta(n)$
- Infine, data una rappresentazione per punti, come valutare il polinomio in un nuovo punto? Si converte prima il polinomio nella rappresentazione per coefficienti e poi si valuta nel nuovo punto.



SCHEMA GRAFICO PER UN EFFICIENTE PROCESSO DI MOLTIPLICAZIONE

- Intervengono le radici n-esime dell'unità.



MOLTIPLICAZIONE RAPIDA DI POLINOMI NELLA FORMA PER COEFFICIENTI

- Si può utilizzare il metodo lineare di moltiplicazione per la rappresentazione per punti per accelerare la moltiplicazione tra rappresentazioni per coefficienti?
- Se scegliamo bene i punti di valutazione, il processo di valutazione e interpolazione può essere realizzato in tempo $\Theta(n \log n)$
- Se scegliamo le radici complesse dell'unità, si può ottenere una rappresentazione per punti considerando la trasformata discreta di Fourier (DFT) di un vettore di coefficienti.
- L'operazione inversa, l'interpolazione, si può fare con la DFT inversa delle coppie di valori, ottenendo un vettore di coefficienti.
- La Fast Fourier Transform (FFT) calcola DFT e DFT inversa in tempo $\Theta(n \log n)$
- *Nota che alcuni testi invertano le due definizioni.*



QUALI STEP?

1. Raddoppiare il grado limite: creare le rappresentazioni per coefficienti di $A(x)$ e $B(x)$ come polinomi di grado limite $2n$, aggiungendo a ciascuna rappresentazione n coefficienti 0 di grado superiore.
2. Valutazione: Calcolare le rappresentazioni per punti di $A(x)$ e $B(x)$ applicando la FFT di ordine $2n$. Queste rappresentazioni contengono i valori dei due polinomi nelle radici $(2n)$ -esime dell'unità.
3. Moltiplicazione punto per punto: calcolare una rappresentazione per punti del polinomio $C(x)=A(x)B(x)$, moltiplicando questi valori punto per punto. La rappresentazione risultante contiene il valore di $C(x)$ in ciascuna radice $(2n)$ -esima.
4. Interpolazione: creare la rappresentazione per coefficienti del polinomio $C(x)$ applicando la FFT a $2n$ coppie di valori per calcolare la DFT inversa

Poiché i passi 1 e 3 richiedono tempo $\Theta(n)$ e i passi 2 e 4 richiedono tempo $\Theta(n \log n)$, il prodotto di due polinomi di grado limite n può essere calcolato in tempo $\Theta(n \log n)$ rappresentando l'input e l'output per coefficienti.



RADICI COMPLESSE DELL'UNITÀ

- Una radice n -esima complessa dell'unità è un numero complesso ω tale che $\omega^n = 1$
- Esistono esattamente n radici n -esime complesse dell'unità:
 $e^{\frac{2\pi i k}{n}}$, per $k=0,1,\dots,n-1$
- Per interpretare questa formula, usiamo la definizione dell'esponenziale di un numero complesso:
$$e^{iu} = \cos(u) + i \sin(u)$$

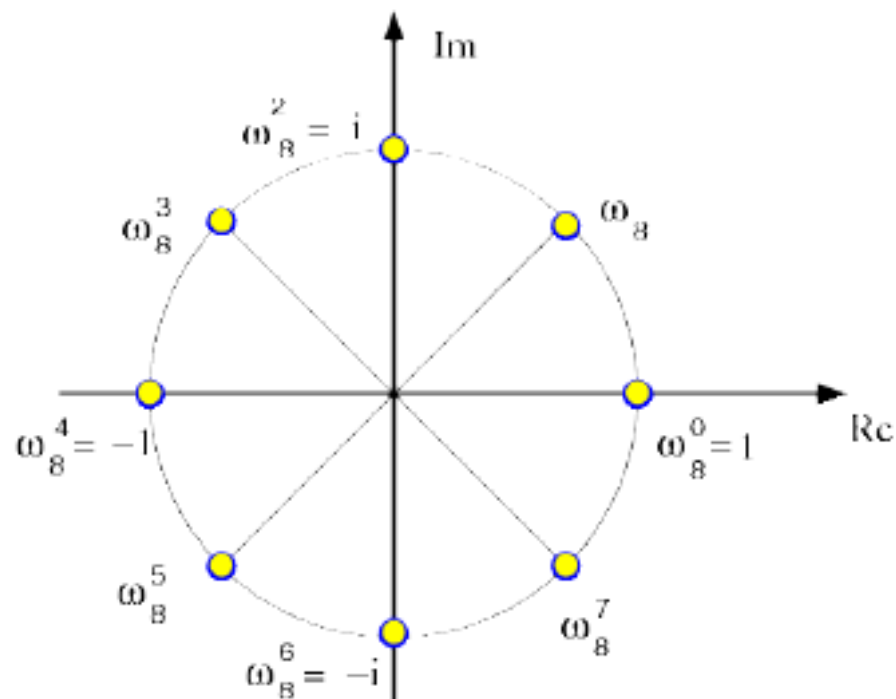
dove i è l'unità immaginaria tale che $i^2 = -1$.

- Le n radici complesse dell'unità sono equamente distanziate nel cerchio di raggio 1 del piano complesso
- Il valore principale $\omega_n = e^{\frac{2\pi i}{n}}$ è detta radice n -esima principale. Le altre sono potenze di ω_n



RAPPRESENTAZIONE SUL PIANO COMPLESSO

- Radici ottave dell'unità



ALCUNE PROPRIETÀ

- **Lemma della cancellazione:** per ogni intero $n \geq 0, k \geq 0$ e $d > 0$

$$\omega_{dn}^{dk} = \omega_n^k$$

- per ogni $n > 0$: (intero pari)

$$\omega_n^{n/2} = \omega_2 = -1$$

- **Lemma del dimezzamento:** se $n > 0$ è pari, i quadrati delle n radici complesse n -esime dell'unità sono le $n/2$ radici $(n/2)$ -esime dell'unità.
- **Lemma della somma:** per ogni $n \geq 1$ ed intero non-negativo k non divisibile per n :

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0$$



TRASFORMATA DISCRETA DI FOURIER

- Si vuole valutare il polinomio $A(x) = \sum_{j=0}^{n-1} a_j x^j$ di grado limite n in $\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}$. Se il polinomio è dato nella forma per coefficienti, definiamo

$$y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{kj}$$

- Il vettore $(y_0, y_1, \dots, y_{n-1})$ è la trasformata discreta di Fourier del vettore dei coefficienti di A , ovvero $y = DFT_n(a)$.
- Attraverso la FFT che sfrutta le proprietà delle radici complesse dell'unità, è possibile calcolare $DFT_n(a)$ in tempo $O(n \log n)$, invece che $O(n^2)$



FAST FOURIER TRANSFORM

- E' un algoritmo efficiente che in tempo $\Theta(n \log n)$ calcola la DFT sfruttando le proprietà delle radici n-esime dell'unità.
- Usa una strategia divide et impera, definendo due sottoproblemi ottenuti considerando i coefficienti pari e dispari di $A(x)$, ovvero i due seguenti polinomi di grado limite $n/2$

$$A^{[0]}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{\frac{n}{2}-1}$$
$$A^{[1]}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{\frac{n}{2}-1}$$

da cui segue che $A(x) = A^{[0]}(x^2) + xA^{[1]}(x^2)$

- Il problema si riduce quindi a calcolare i due polinomi in $(\omega_n^0)^2, (\omega_n^1)^2, \dots, (\omega_n^{n-1})^2$ e poi combinare i risultati, utilizzando la precedente relazione.
- Per il lemma del dimezzamento, la lista di valori non è composta da n elementi distinti ma ciascuna radice si presenta due volte. Quindi i polinomi $A^{[0]}(x)$ e $A^{[1]}(x)$ sono calcolati ricorsivamente dalle $n/2$ radici $n/2$ esime dell'unità. Ovvero, si deve calcolare $DFT_{\frac{n}{2}}$.
Supponiamo che n sia una potenza di 2.



FFT

Recursive-FFT(a)

1. $n = a.length$
2. If $n == 1$
3. return a
4. $\omega_n = e^{\frac{2\pi i}{n}}$
5. $\omega = 1$
6. $a^{[0]} = (a_0, a_2, \dots, a_{n-2})$
7. $a^{[1]} = (a_1, a_3, \dots, a_{n-1})$
8. $y^{[0]} = \text{Recursive-FFT}(a^{[0]})$
9. $y^{[1]} = \text{Recursive-FFT}(a^{[1]})$
10. For $k=0$ to $n/2-1$
11. $y_k = y_k^{[0]} + \omega y_k^{[1]}$
12. $y_{k+n/2} = y_k^{[0]} - \omega y_k^{[1]}$
13. $\omega = \omega \omega_n$
14. return y

Il tempo di esecuzione è $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$



INTERPOLAZIONE DELLE RADICI COMPLESSE

- Si interpolano le radici complesse con un polinomio, ovvero si passa dalla rappresentazione per punti a quella per coefficienti.
- Usiamo la matrice di Vandermonde $V_n = (\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1})$ e si utilizza l'inversa di questa matrice.
- Teorema: Per $j, k=0, \dots, n-1$, l'elemento di posto (j, k) in V_n^{-1} è ω_n^{-kj}/n
- Data la matrice inversa V_n^{-1} , abbiamo che la $DFT_n^{-1}(y)$ è
$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-kj}$$
- Si può verificare facilmente che si può utilizzare lo stesso algoritmo della FFT scambiando i ruoli di a e y , sostituendo ω_n con ω_n^{-1} e dividendo ciascun elemento del risultato per n



ESERCIZIO

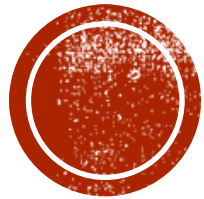
- Scrivere un programma -----> che calcola la FFT



ESERCIZIO 1

- Calcolare $\text{DFT}(1, 2, 3, 4)$.
- Calcolare $\text{DFT}(i, -2i, 5i, -4i)$





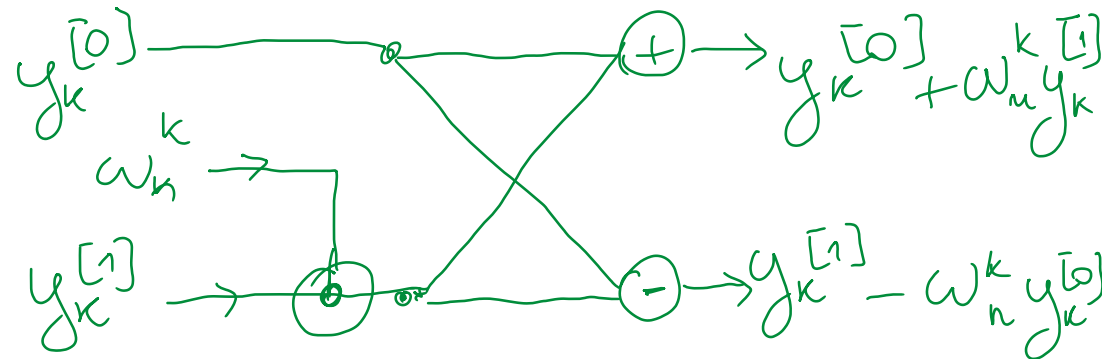
IMPLEMENTAZIONI EFFICIENTI DELLA FFT

FFT

Recursive-FFT(a)

1. $n = a.length$
2. If $n == 1$
3. return a
4. $\omega_n = e^{\frac{2\pi i}{n}}$
5. $\omega = 1$
6. $a^{[0]} = (a_0, a_2, \dots, a_{n-2})$
7. $a^{[1]} = (a_1, a_3, \dots, a_{n-1})$
8. $y^{[0]} = \text{Recursive-FFT}(a^{[0]})$
9. $y^{[1]} = \text{Recursive-FFT}(a^{[1]})$
10. For $k=0$ to $n/2-1$
 11. $y_k = y_k^{[0]} + \omega y_k^{[1]}$
 12. $y_{k+n/2} = y_k^{[0]} - \omega y_k^{[1]}$
 13. $\omega = \omega \omega_n$
14. return y

OPERAZIONE BUTTERFLY



For $k=0$ to $n/2-1$

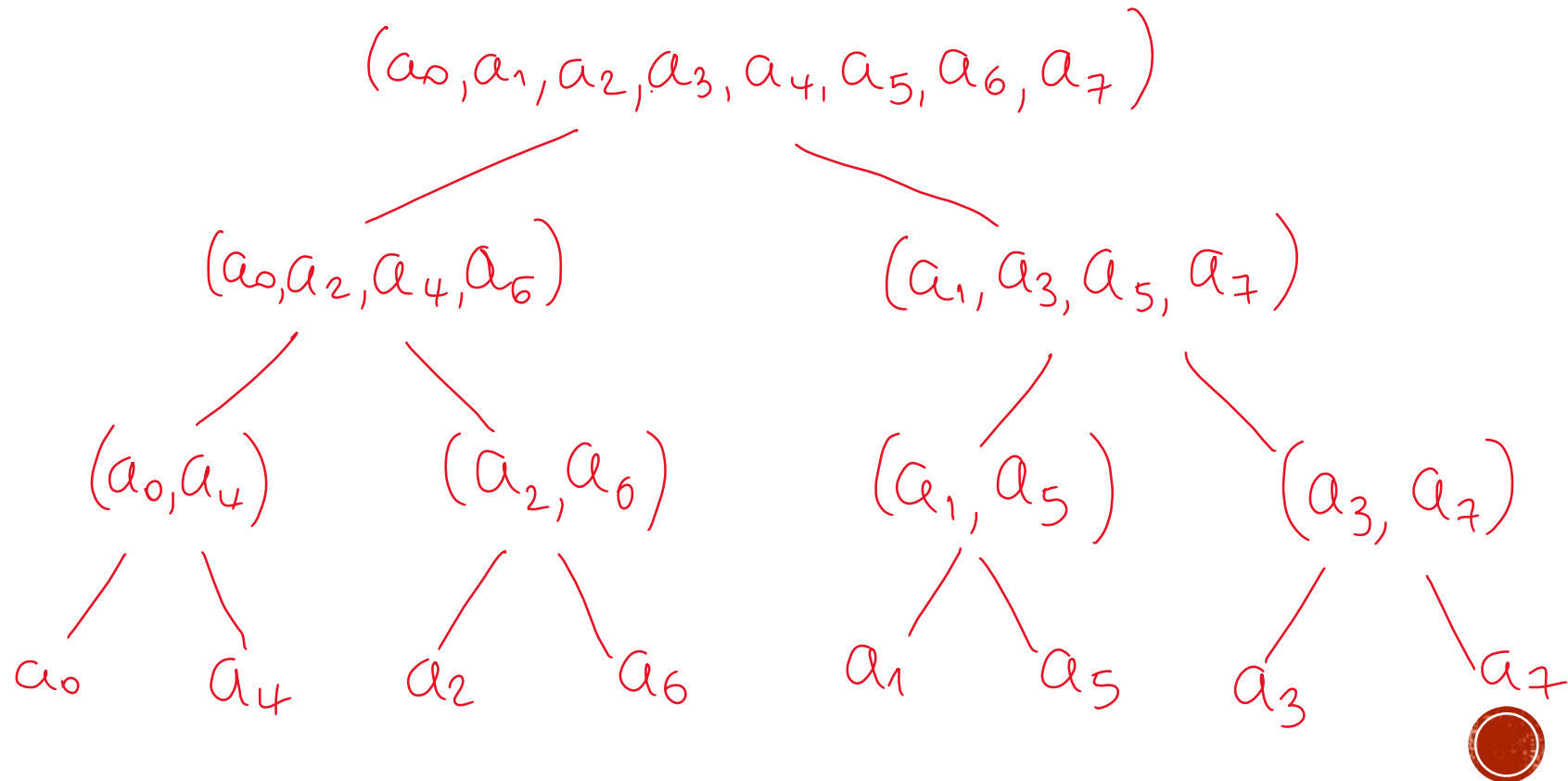
$$\begin{aligned}
 t &= \omega y_k^{[1]} \\
 y_k &= y_k^{[0]} + t \\
 y_{k+n/2} &= y_k^{[0]} - t \\
 \omega &= \omega \omega_n
 \end{aligned}$$

Il tempo di esecuzione è $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$



IMPLEMENTAZIONE ITERATIVA DELLA FFT

- Consideriamo l'albero delle chiamate ricorsive



IMPLEMENTAZIONE ITERATIVA DELLA FFT

- Se disponiamo gli elementi nello stesso ordine in cui compaiono nelle foglie dell'albero, possiamo eseguire la procedura ricorsiva partendo dal basso, calcolando la DFT di ogni coppia utilizzando un'operazione butterfly, e sostituendo la coppia con la sua DFT. E così via.

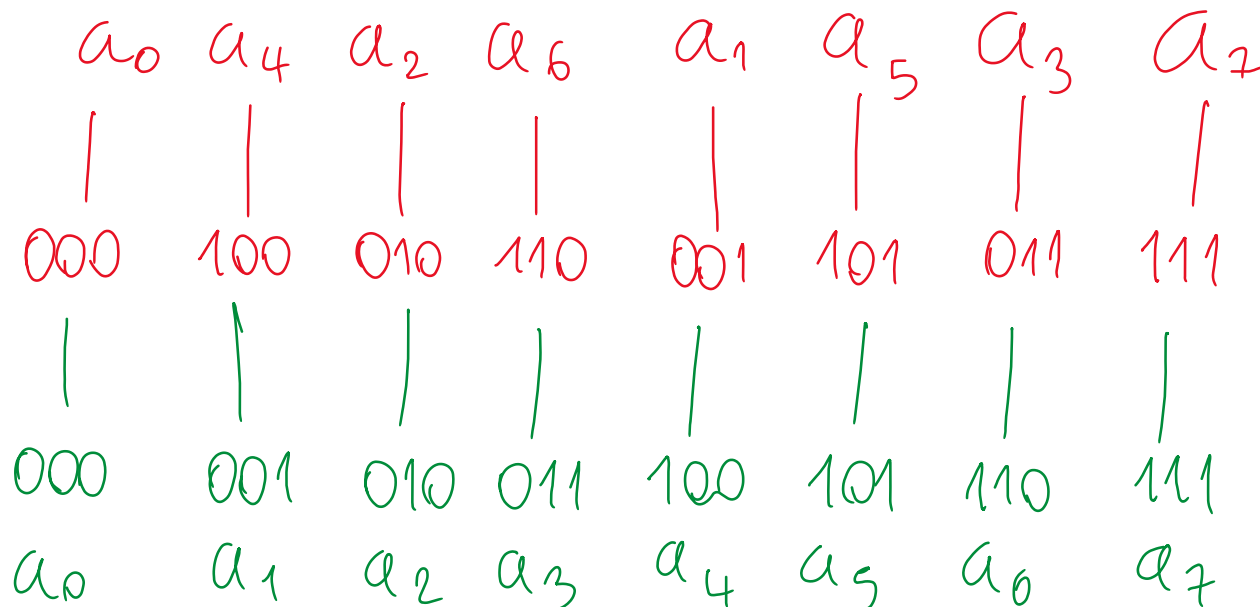
- Servono $\log n$ livelli:

1. **for** $s \leftarrow 1$ **to** $\lg n$ **do**
2. **for** $k \leftarrow 0$ **to** $n - 1$ **by** 2^s **do**
3. combine the two 2^{s-1} -element DFT's in
4. $A[k..k + 2^{s-1} - 1]$ and $A[k + 2^{s-1}..k + 2^s - 1]$
5. into one 2^s -element DFT in $A[k..k + 2^s - 1]$



IMPLEMENTAZIONE ITERATIVA DELLA FFT

- Supponiamo di avere un array A in cui gli elementi sono disposti nello stesso ordine delle foglie dell'albero
- Combiniamo gli elementi dal basso verso l'alto e applichiamo la DFT



Invertiamo
i bit



IMPLEMENTAZIONE ITERATIVA DELLA FFT

- Se si considera l'albero dalla radice alle foglie, ad ogni livello gli indici con il bit meno significativo a 0 sono posti a sinistra, quelli con il bit a 1 sono posti a destra. Al livello successivo si ripete lo stesso ragionamento escludendo il bit meno significativo. Arrivati alle foglie si ottiene la permutazione in cui l'ordine è dato per inversione dei bit.



ITERATIVE FFT

Iterative-FFT(a)

1. BIT-REVERSE-COPY(a,A)
2. $n = a.length$
3. For $s=1$ to $\log n$
4. $m = 2^s$
5. $\omega_n = e^{\frac{2\pi i}{m}}$
6. For $k=0$ to $n-1$ by m
7. $\omega = 1$
8. For $j=0$ to $m/2 - 1$
9. $t = \omega A\left[k + j + \frac{m}{2}\right]$
10. $u = A[k + j]$
11. $A[k + j] = u + t$
12. $A\left[k + j + \frac{m}{2}\right] = u - t$
13. $\omega = \omega \omega_n$

BIT-REVERSE-COPY(a,A)

$n = a.length$

For $k=0$ to $n-1$

$$A[rev(k)] = a_k$$

$O(n \log n)$

Il tempo di esecuzione è $T(n) = \Theta(n \log n)$



ESEMPIO DI PROGRAMMA PER INVERTIRE I BIT DI UN DATO NUMERO

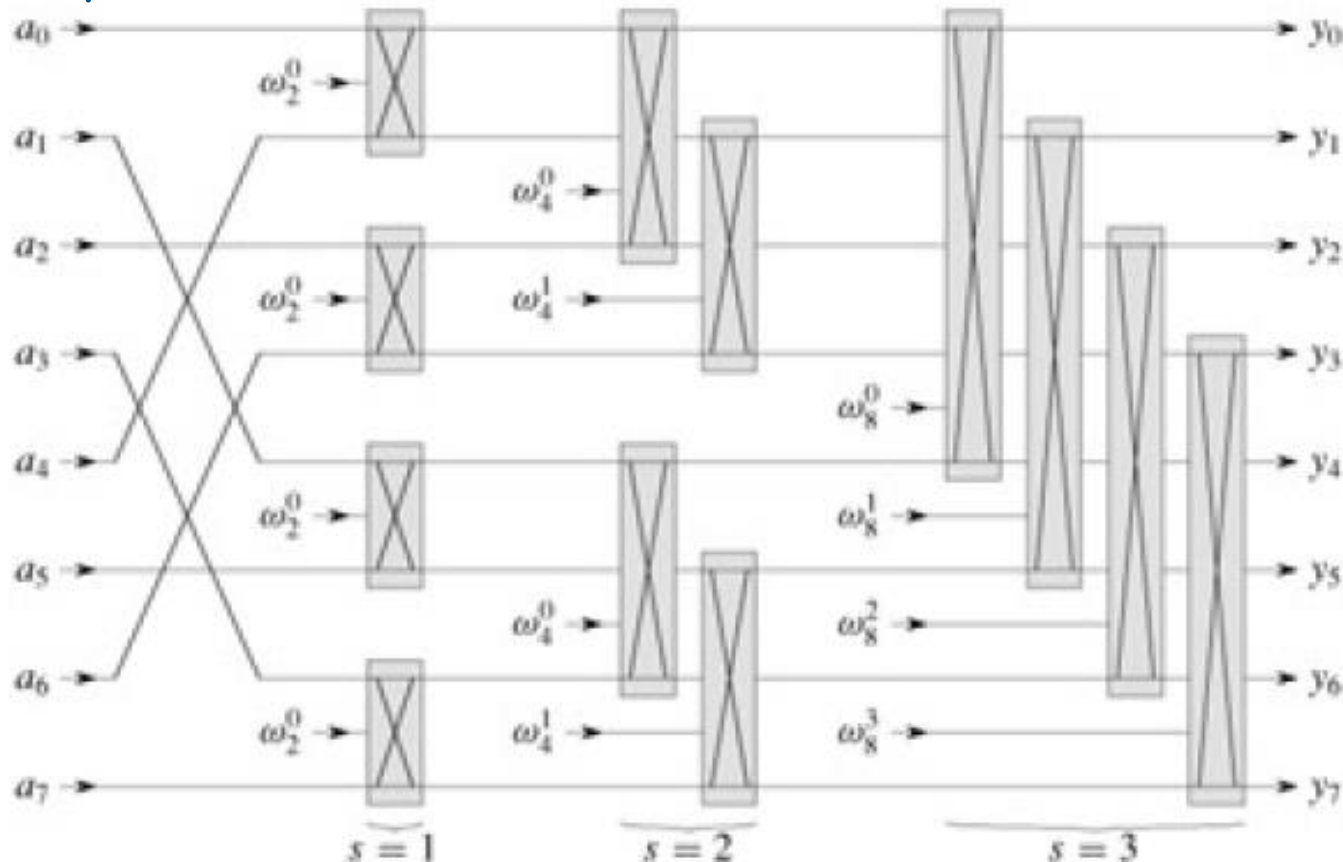
```
reverseBits(n)
  Initialize rev = 0

  while (n > 0)
    rev <<= 1
    if n & 1 == 1, then
      rev ^= 1
    n >>= 1

  return rev
```



CIRCUITO PARALLELO PER FFT



Le operazioni butterfly che vengono utilizzate sono indipendenti, allora potrebbero essere eseguite in parallelo.



ESERCIZIO 1

- Calcolare $\text{DFT}(1, 2, 3, 4)$.
- Calcolare $\text{DFT}(i, -2i, 5i, -4i)$



ESERCIZIO 2

- Dato il vettore $a = (1 + i, -3, 5 + 7i, -2)$, chi è $\text{DFT}(a)$?
- Dato il vettore $b = (3 + 4i, 4, 7, 1 - i)$, chi è $\text{DFT}(b)$?
- Calcolare $\text{DFT}(a+b)$



ESERCIZIO 3

- Come calcolare il prodotto dei polinomi:
 $p(x) = 1 + 5x + 17x^2 + 2x^3$ e $q(x) = 11 + 6x - 4x^2 + 5x^3$
- Scrivere un programma che dati due polinomi a coefficienti interi ne calcoli il prodotto.
- Siano dati i due polinomi $p(x) = 1 + 5x + 17x^2$ e $q(x) = 11 + 6x - 4x^2$, calcolare il polinomio prodotto usando la FFT.





TRAFORMATA DI FOURIER

ESERCIZI

1. Scrivere un programma in Java che calcola la trasformata di Fourier

ESERCIZI

// Funzione ricorsiva FFT

```
public static Complex[] fft(Complex[] input) {  
    int n = input.length;
```

// base case

```
if (n == 1) return new Complex[] { input[0] };    CASO BASE: C'E' UN SOLO ELEMENTO
```

```
if (n % 2 != 0) throw new IllegalArgumentException("Input length must be a power of 2");  
RICHIEDE LA POTENZA DEL 2
```

// Divide: FFT delle parti pari e dispari

```
Complex[] even = new Complex[n / 2];
```

```
Complex[] odd = new Complex[n / 2];
```

```
for (int i = 0; i < n / 2; i++) {
```

```
    even[i] = input[2 * i];
```

SEPARO GLI ELEMENTI CON INDICE PARI E DISPARI

```
    odd[i] = input[2 * i + 1];
```

```
}
```

CALCOLO LA RICORSIVA DELLA FFT

```
Complex[] fftEven = fft(even);
```

```
Complex[] fftOdd = fft(odd);
```

// COMBINA

USA IL fattore rotante $w = e^{-2\pi i k / N}$ $w = e^{-2\pi i k / N}$

```
Complex[] result = new Complex[n];
```

```
for (int k = 0; k < n / 2; k++) {
```

```
    double angle = -2 * Math.PI * k / n;
```

```
    Complex w = new Complex(Math.cos(angle), Math.sin(angle));
```

```
    Complex wkOdd = w.mul(fftOdd[k]);
```

```
    result[k] = fftEven[k].add(wkOdd);
```

```
    result[k + n/2] = fftEven[k].sub(wkOdd);
```

```
}
```

```
return result;
```

```
}
```

ESERCIZI

1. Scrivere un programma in Java che calcola $DFT(1,2,3,4)$

Per K=0

$$X_0 = x_0 + x_1 + x_2 + x_3 = 1 + 2 + 3 + 4 = 10$$

Per K=1

$$X_1 = x_0 \cdot e^{-2\pi i \cdot 0/4} + x_1 \cdot e^{-2\pi i \cdot 1/4} + x_2 \cdot e^{-2\pi i \cdot 2/4} + x_3 \cdot e^{-2\pi i \cdot 3/4}$$

Usiamo: $W = e^{-2\pi i/4} = i^{-1} = -i$

$$X_1 = 1 \cdot 1 + 2 \cdot (-i) + 3 \cdot (-1) + 4 \cdot i = 1 - 2i - 3 + 4i = (-2) + 2i$$

Per K=2

$$X_2 = x_0 + x_1 \cdot e^{-2\pi i \cdot 2/4} + x_2 \cdot e^{-2\pi i \cdot 4/4} + x_3 \cdot e^{-2\pi i \cdot 6/4}$$

$$X_2 = 1 + 2 \cdot (-1) + 3 \cdot 1 + 4 \cdot (-1) = 1 - 2 + 3 - 4 = -2$$

Per K=3

$$X_3 = x_0 + x_1 \cdot e^{-2\pi i \cdot 3/4} + x_2 \cdot e^{-2\pi i \cdot 6/4} + x_3 \cdot e^{-2\pi i \cdot 9/4}$$

$$X_3 = 1 + 2 \cdot i + 3 \cdot (-1) + 4 \cdot (-i) = 1 + 2i - 3 - 4i = -2 - 2i$$

$$\text{DFT}(1, 2, 3, 4) = \begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{bmatrix} = \begin{bmatrix} 10 \\ -2 + 2i \\ -2 \\ -2 - 2i \end{bmatrix}$$

ESERCIZI

1. Scrivere un programma in Java che calcola DFT($i, -2i, 5i, -4i$)

$$x_0 = i, \quad x_1 = -2i, \quad x_2 = 5i, \quad x_3 = -4i$$

Convertiamo in forma esplicita

$$x_0 = (0 + 1i), \quad x_1 = (0 - 2i), \quad x_2 = (0 + 5i), \quad x_3 = (0 - 4i)$$

Si calcola per $k=0,1,2,3$ come prima

$$\text{DFT}(i, -2i, 5i, -4i) = \begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 - 4i \\ 12i \\ -2 - 4i \end{bmatrix}$$

Dato il vettore $\mathbf{a} = (1 + i, -3, 5 + 7i, -2)$, chi è $\text{DFT}(\mathbf{a})$?

$$\mathbf{a} = (1 + i, -3, 5 + 7i, -2)$$

$$\mathbf{b} = (3 + 4i, 4, 7, 1 - i)$$

$$\mathbf{a} + \mathbf{b} = (1 + i + 3 + 4i, -3 + 4, (5 + 7i) + 7, -2 + (1 - i))$$

$$\mathbf{a} + \mathbf{b} = (4 + 5i, 1, 12 + 7i, -1 - i)$$

$$\mathbf{a} = (1 + i, -3, 5 + 7i, -2)$$

$$A_0$$

$$A_0 = (1 + i) + (-3) + (5 + 7i) + (-2) = (1 - 3 + 5 - 2) + (i + 7i) = 1 + 8i$$

$$A_1$$

$$\begin{aligned} A_1 &= (1 + i) \cdot 1 + (-3)(-i) + (5 + 7i)(-1) + (-2)(i) \\ &= (1 + i) + 3i + (-5 - 7i) + (-2i) = (1 - 5) + (i + 3i - 7i - 2i) = -4 - 5i \end{aligned}$$

$$A_2$$

$$A_2 = (1 + i) + (-3)(-1) + (5 + 7i)(1) + (-2)(-1) = (1 + i) + 3 + (5 + 7i) + 2 = (1 + 3 + 5 + 2) + (i + 7i) = 11 + 8i$$

$$A_3$$

$$A_3 = (1 + i) + (-3)(i) + (5 + 7i)(-1) + (-2)(-i) = (1 + i) - 3i - (5 + 7i) + 2i = (1 - 5) + (i - 3i - 7i + 2i) = -4 -$$



Dato il vettore $b = (3 + 4i, 4, 7, 1 - i)$, chi è $DFT(b)$?

$$B_0$$

$$B_0 = (3 + 4i) + 4 + 7 + (1 - i) = (3 + 4 + 7 + 1) + (4i - i) = 15 + 3i$$

$$B_1$$

$$B_1 = (3 + 4i)(1) + 4(-i) + 7(-1) + (1 - i)(i)$$

- $4(-i) = -4i$
- $7(-1) = -7$
- $(1 - i)(i) = i - i^2 = i + 1$

$$B_1 = (3 + 4i) - 4i - 7 + (1 + i) = (3 - 7 + 1) + (4i - 4i + i) = -3 + i$$

$$B_2$$

$$B_2 = (3 + 4i) + 4(-1) + 7 + (1 - i)(-1) = (3 + 4i) - 4 + 7 - (1 - i) = (3 - 4 + 7 - 1) + (4i + i) = 5 + 5i$$

$$B_3$$

$$B_3 = (3 + 4i) + 4(i) + 7(-1) + (1 - i)(-i)$$



Calcolare DFT(a+b)

k	DFT(a)	DFT(b)	Somma
0	$1 + 8i$	$15 + 3i$	$16 + 11i$
1	$-4 - 5i$	$-3 + i$	$-7 - 4i$
2	$11 + 8i$	$5 + 5i$	$16 + 13i$
3	$-4 - 7i$	$-5 + 7i$	$-9 + 0i = -9$



ITERATIVE-FFT

```
static void iterativeFFT(Complex[] a) {  
    int n = a.length;  
    Complex[] A = new Complex[n];  
    bitReverseCopy(a, A);  
  
    for (int s = 1; s <= Integer.numberOfTrailingZeros(n); s++) {  
        int m = 1 << s;  
        double angle = -2 * Math.PI / m;  
        Complex wm = new Complex(Math.cos(angle),  
Math.sin(angle));  
  
        for (int k = 0; k < n; k += m) {  
            Complex w = new Complex(1, 0);  
            for (int j = 0; j < m / 2; j++) {  
                Complex t = w.mul(A[k + j + m / 2]);  
                Complex u = A[k + j];  
                A[k + j] = u.add(t);  
                A[k + j + m / 2] = u.sub(t);  
                w = w.mul(wm);  
            }  
        }  
    }  
}
```



Invertire i bit di un dato numero

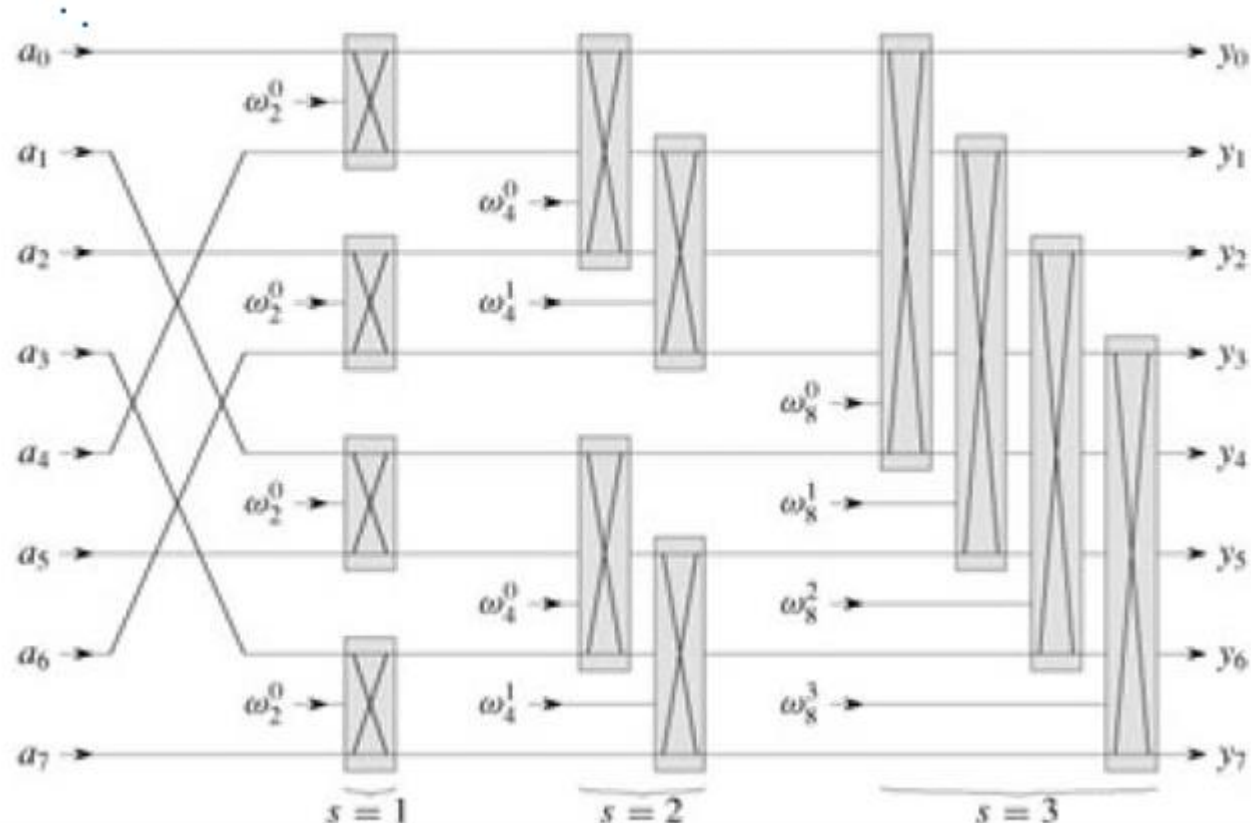
```
static void bitReverseCopy(Complex[] a, Complex[] A) {  
    int n = a.length;  
    int logN = Integer.numberOfTrailingZeros(n);  
    for (int k = 0; k < n; k++) {  
        int rev = Integer.reverse(k) >>> (32 - logN);  
        A[rev] = a[k];  
    }  
}
```



Un circuito parallelo per la FFT (Fast Fourier Transform) è un modello hardware (o schema teorico) che esegue la FFT sfruttando la parallelizzazione dei calcoli. Questo tipo di circuito è utile in architetture parallele (come GPU, FPGA, DSP) per ridurre il tempo di calcolo da $O(n \log n)$ seriale a un tempo logaritmico $O(\log n)$ con n processori.



CIRCUITO PARALLELO PER FFT



Le operazioni butterfly che vengono utilizzate sono indipendenti, allora potrebbero essere eseguite in parallelo

